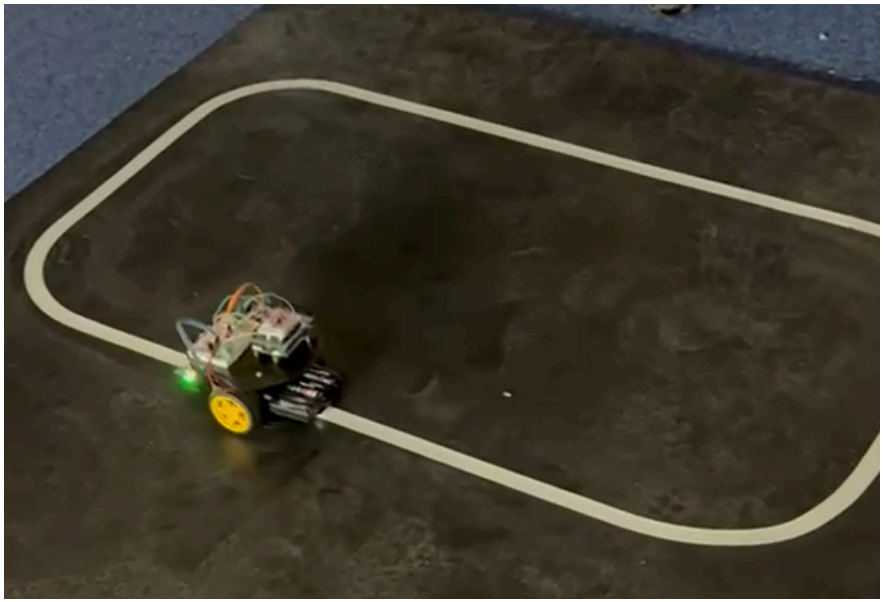
A Line-Follow Robot

As part of a design module we were giving a set of challenges centered around getting a small robot to follow instructions such as:

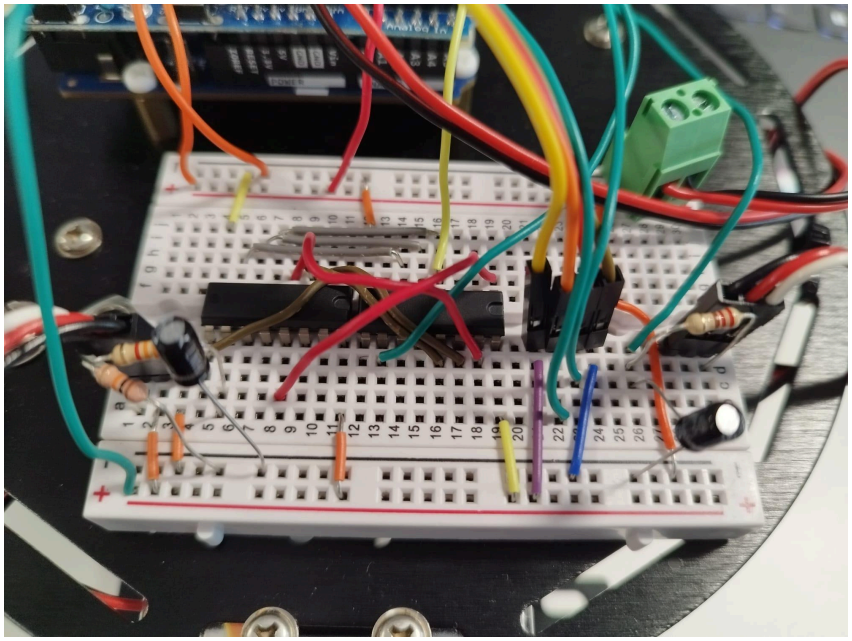
- Follow a line.
- Move forward/backwards some specified distance.
- Turn left/right at a specified angle.
- Follow the line whilst conforming to a specified speed profile.
- All using a WiFi network & UI-side control.

I was personally responsible for the entirety of the coding and electronics of the project (the two other group members focused on creating reports, presentations, & videos around it).

There were a whole host of problems associated with building the robot over the 12 weeks but I'll zone-in on how getting it to follow the line at a specific speed profile worked because few others managed to achieve that and my approach stretched well outside of the module content.

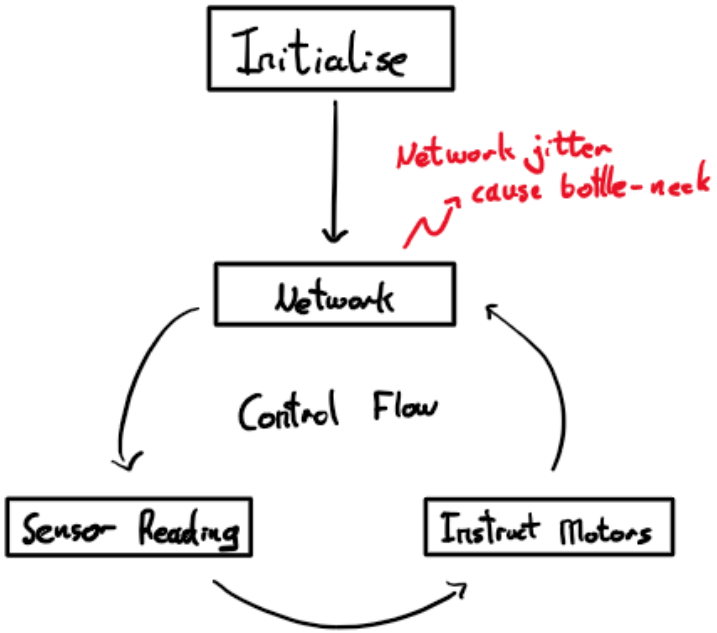


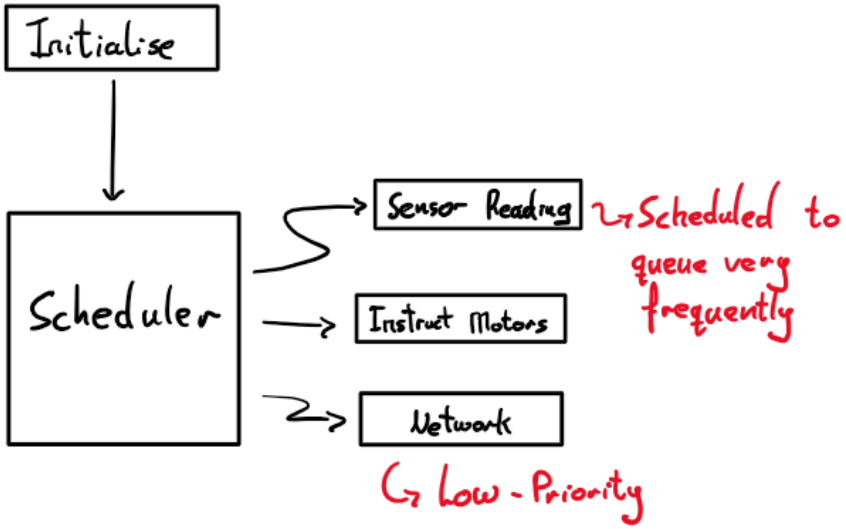
Robot on the Line



Breadboard Circuit

https://github.com/Adam76e54/GoldChallenge
https://github.com/Adam76e54/SilverChallenge
https://github.com/Adam76e54/BronzeChallenge





Super Loop **Real-Time Operating System**

The principal problem was recognising it was time for a real-time operating system (RTOS). The bot kept coming off the track when given high speeds. We were initially using super-loop logic for the firmware: repeatedly cycling through steps such as “read from network”, “take sensor inputs”, “execute network commands”. That approach becomes invalid if you need to be repeatedly executing a time-critical task in a deterministic way. Basic testing using the Arduino library micros() showed that, occasionally, network jitter was eating up CPU time and lowering the frequency of infrared sensor readings which the robot relied upon to stay on track.
After some research and learning about how real-time operating systems worked, I rejigged the code-base to use the Arduino R4’s port of the FreeRTOS (API for the C language). This allowed the robot to stay on track whilst using a PID control loop to conform to any speed profile it was given.

https://github.com/Adam76e54/GoldChallenge
https://github.com/Adam76e54/SilverChallenge
https://github.com/Adam76e54/BronzeChallenge

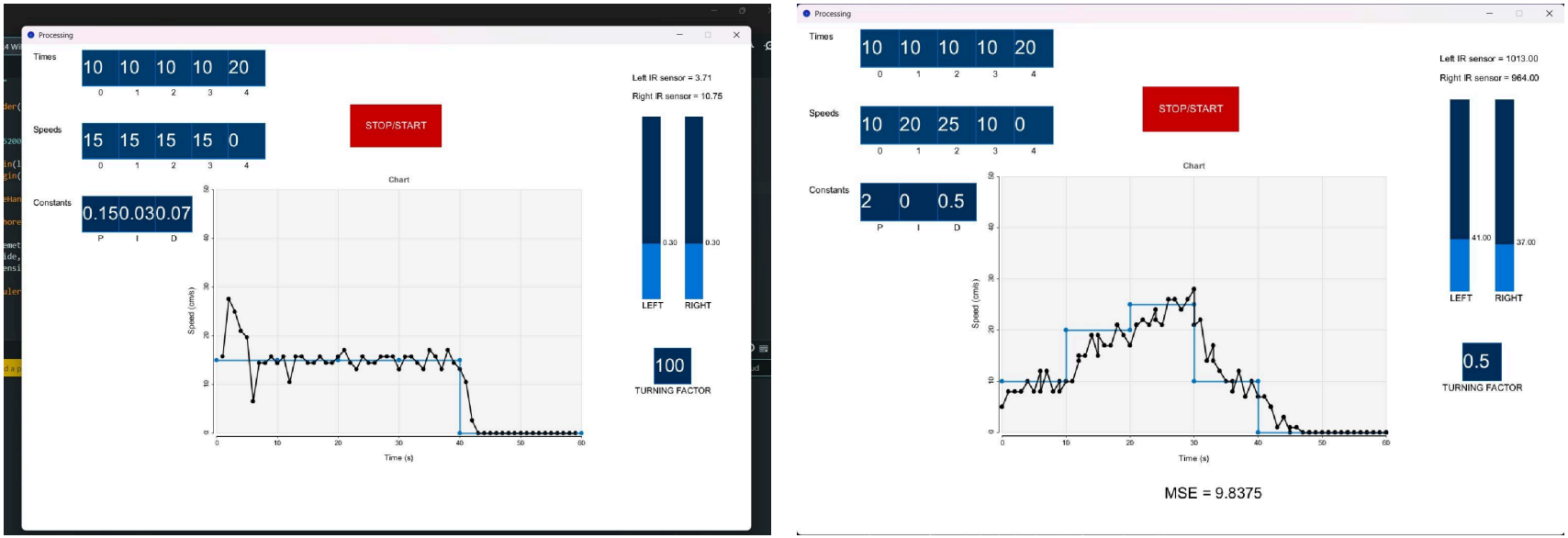
Some other features of the design included:

Electronics:

- RC filters on wheel encoder outputs to track wheel motion.
- Binary counters and shift register IC's to track wheel motion (this could've been done in code but the module constrained us to using circuitry for this).
- A motor driver IC for driving the motors.

Software:

- A custom built FiFo buffer for processing packets.
- Custom drivers for electronic components such as the ultrasonic sensor & each of the ICs.
- A 'Controller' object that organised PID feedback control for steady speed and wheel matching.
- A user interface built using Processing (a java-like platform)



Screenshot of UI & target vs actual speed profiles

https://github.com/Adam76e54/GoldChallenge
https://github.com/Adam76e54/SilverChallenge
https://github.com/Adam76e54/BronzeChallenge